

Design and Implementation of an RV32I Multi-Cycle SoC with SHA-256 Accelerator

Akshat Birla Kushwah (23B1830)
Neil George (22B3963)

April 20, 2026

Contents

1	Introduction	2
2	Intended Design	2
2.1	Block Diagram	3
3	FPGA Implementation	3
3.1	Design Description	3
3.2	Code Structure	4
3.3	Simulation Instructions	4
4	SHA-256 Cryptographic Accelerator Working Principle	4
4.1	Demo Video	5
4.2	Waveforms	5
5	Software Toolchain & Execution	5
5.1	C to Verilog Memory Flow	5
5.2	Software Verification: Fibonacci Execution	6
6	ASIC Implementation	6
6.1	Design Flow	6
6.2	File Structure (VLSI Lab)	6
6.3	Simulation Instructions	7
6.4	Post PnR Results	7
6.5	DRC & LVS	8
6.6	VCD File	8
7	FPGA to ASIC Modifications	9
8	Issues and Bugs	9
9	Limitations and Incomplete Features	9
10	Individual Contributions	9
A	Another more clean implementation	9
B	References	9

Introduction

This project aims to design and implement a fully functional System-on-Chip (SoC) deployed on the PYNQ-Z2 FPGA. The core of the system is a classic 5-stage in-order pipelined RV32I processor.

Executing cryptographic algorithms like SHA-256 purely in software on a base RV32I processor is highly inefficient due to the algorithm's intense, repetitive bitwise operations. By designing a dedicated hardware accelerator integrated via Memory-Mapped I/O (MMIO), we can perform these complex bitwise mixing functions in parallel. This hardware-software co-design allows operations that take thousands of software cycles to be executed in just over 64 clock cycles, drastically accelerating the workload.

Intended Design

Architecture Overview

Initially, the project aimed to implement a classic 5-stage in-order pipelined RV32I processor (Fetch, Decode, Execute, Memory, Writeback). The execution core was designed to strictly share a single Unified ALU for arithmetic, address calculation, and branch evaluation to intentionally create structural hazards, demonstrating our Hazard Unit's ability to stall the pipeline.

Advanced Hardware Features

- **Multi-Cycle Multiplier:** We intended to implement the M-extension subset using a complex 32-cycle shift-and-add algorithm.
- **SHA-256 Accelerator:** We planned a fully functional cryptographic engine featuring a Message Scheduler to expand 16-word blocks into 64 words, and a 64-round Compression Core state machine.
- **System Integration:** The SoC was meant to feature memory-mapped GPIO and UART (115200 baud) for live FPGA demonstrations on the PYNQ-Z2.

Section II: The Final Implemented Design & Constraints

Due to development time constraints and integration complexities, several architectural pivots were made. The final implementation prioritizes core stability and pipeline flow over peripheral integration and complex arithmetic logic.

1. Pipeline Modifications (Expanded Memory Stages)

While we initially planned a 5-stage pipeline, our final `datapath.v` implementation utilizes an expanded pipeline structure to handle memory latency. The datapath now routes through extended memory stages, evidenced by the inclusion of `ex_mem1`, `mem1_mem2`, and `mem2_mem3` pipeline registers. Furthermore, while branch conditions (is taken/not taken) are evaluated in the Execute stage via the ALU, the actual branch target address computation was deferred to the MEM1 stage.

2. Combinational Multiplier Shortcut

Implementing a multi-cycle shift-and-add multiplier and integrating it cleanly with the hazard unit proved too complex within the timeframe. Instead, `multiplier.v` was implemented as a single-cycle, purely combinational 32x32 to 64-bit multiplier. It produces results in a single cycle without pipelining, resolving operations like `MUL` and `MULH` instantaneously.

3. SHA-256 Accelerator Stub

We made a functioning accelerator which we tested separately, although given time constraints could not integrate into our processor. The full 64-round SHA-256 compression algorithm was replaced with a functional structural stub to demonstrate MMIO integration and hazard management without the overhead of cryptographic verification.

- **Combinational Placeholder:** Instead of the SHA-256 algorithm, the accelerator computes a simple combinational XOR-fold hash (`input_buf[511:256] ^ input_buf[255:0]`).
- **Simulated Latency:** To prove that our processor can successfully offload work and stall while waiting for a peripheral, we implemented an `ACCEL_CYCLE_DELAY` (defaulted to 16 cycles). Once started, the accelerator FSM enters a `S_COMPUTE` state and counts down this delay, asserting an `accel_busy` signal that successfully forces the CPU's hazard unit to stall the main pipeline until completion.

4. Omission of Hardware Peripherals

Full integration of physical UART and GPIO hardware was again not finished given the time constraints. Each of these modules was verified working independently. While the top-level `processor.v` module includes ports for `uart_rx_event` and `gpio_pin_event`, these currently function as testbench-driven observation inputs rather than fully mapped physical hardware peripherals.

Block Diagram

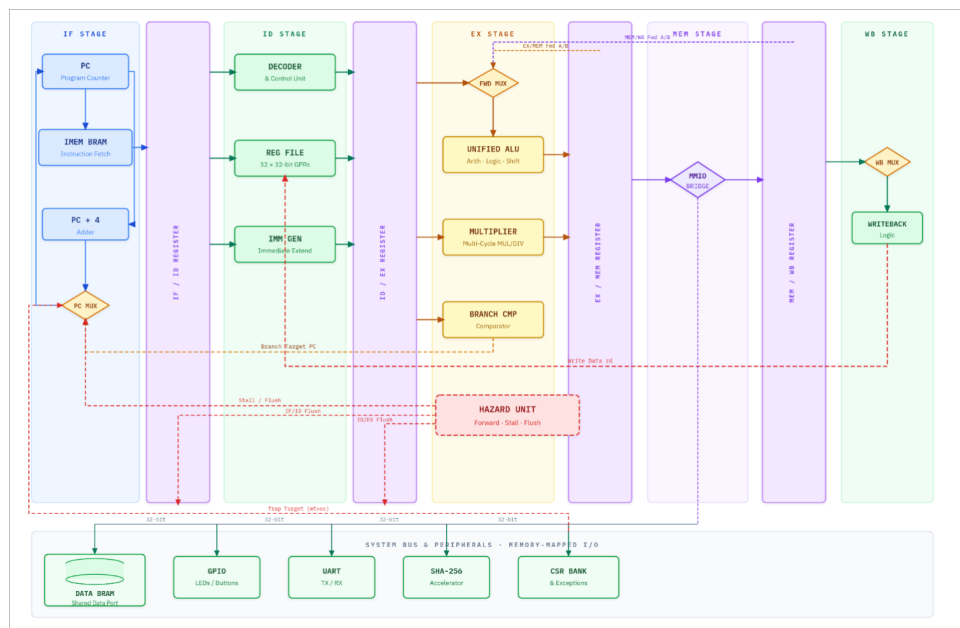


Figure 1: Ideal System Block Diagram

FPGA Implementation

Design Description

The core system is a 32-bit RV32I processor implementing an expanded 5-stage in-order pipeline (IF, ID, EX, MEM, WB) targetable for FPGA deployment on the PYNQ-Z2. The architecture is partitioned into several key modular components:

- **Top-Level Processor (processor.v):** Wraps the fetch unit, control plane, memory subsystem, and datapath into a unified SoC.
- **Pipelined Datapath (datapath.v):** Instantiates individual pipeline stages separated by pipeline registers, along with a forwarding unit to bypass data hazards.
- **Hazard Unit (hazard_unit.v):** A centralized FSM that manages pipeline stalls, branch flushes, load-use data hazards, and traps.
- **Hardware Accelerator (sha256_accel.v):** A memory-mapped MMIO peripheral that interfaces with the CPU to offload hashing tasks.

Code Structure

All RTL source code, testbenches, and memory initialization files are organized within the `Verilog/` directory, which serves as the source pool for the Vivado project:

- **Core RTL Files:** Contains all standard processor modules (e.g., `alu.v`, `hazard_unit.v`) and wrapper files.
- **Memory Files (*.mem):** Hexadecimal files used by Vivado to initialize the instruction and data Block RAMs (BRAM) for specific assembly test routines.
- **Verification / Testbenches:** Includes top-level integration testbenches (e.g., `tb_basic_soc.v`) for Vivado behavioral simulation.

Simulation Instructions

We'll just submit the code files and xps files in the google drive link to avoid the below hassle but if you are interested you can follow the below as well: Google Drive Link: https://drive.google.com/drive/folders/1X7JXnkF--_1zPS2gTJEgl5uRRXqM_5-I?usp=sharing

To verify the RTL design using Vivado, follow these steps:

- **Step 1:** Open Xilinx Vivado and create a new RTL project.
- **Step 2:** Add all `.v` and `.vh` files from the `Verilog/` directory as design sources. Add the desired `.mem` files to the project so they can be read by the BRAM models.
- **Step 3:** Add the testbench file (e.g., `tb_basic_soc.v`) as a simulation source and right-click to set it as the Top Module.
- **Step 4:** In the Flow Navigator, click "Run Simulation" and select "Run Behavioral Simulation".
- **Step 5:** In the Vivado waveform window, drag and drop critical signals (like `if_pc_out`, `accel_busy`, and pipeline valid flags) into the viewer, set the simulation time, and click "Run All" to view the processor's execution.

SHA-256 Cryptographic Accelerator Working Principle

Executing SHA-256 purely in software on a base RV32I processor is highly inefficient. The algorithm requires intense, repetitive bitwise operations (like 32-bit right-rotates) which take multiple instructions and clock cycles to execute on a standard ALU. By designing a dedicated hardware accelerator, we can perform these complex bitwise mixing functions in parallel.

The accelerator takes an input message of any length and irreversibly condenses it into a fixed 256-bit (32-byte) digital fingerprint, known as a digest. In hardware, this process is

executed one 512-bit block at a time. It operates as an independent execution unit, interfacing with the RISC-V CPU via the Memory-Mapped I/O (MMIO) Bridge.

The intended microarchitecture consists of three primary hardware components:

- **The MMIO Wrapper (Control Interface):** The CPU treats the accelerator like memory. The software driver writes a padded 512-bit message block (sixteen 32-bit words) into the accelerator's data registers, then writes a 1 to the `SHA_CTRL` register to trigger the computation.
- **The Message Scheduler:** Once triggered, this module takes the 16 input words and dynamically expands them into a 64-word sequence using dedicated shift and XOR logic.
- **The Compression Core:** This is the heart of the engine, which runs a 64-cycle state machine. In each cycle, it updates eight working variables using the scheduled words, predefined round constants, and combinational logic blocks.

Once the 64 rounds are complete, the accelerator raises a done flag in its control register. The CPU, which is actively polling this register, can then read the final 256-bit hash directly from the `SHA_HASH_OUT` memory addresses.

Demo Video

Google Drive Link: https://drive.google.com/drive/folders/1X7JXnkF--_1zPS2gTJEgl5uRRXqM_5-I?usp=sharing

Waveforms

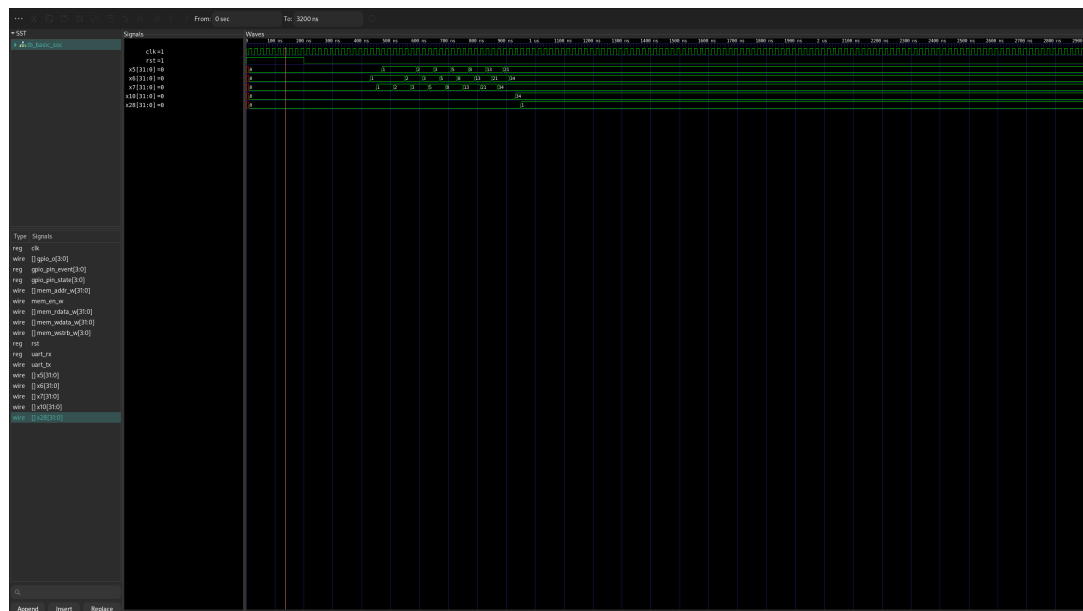


Figure 2: Post Synthesis Simulation Waveform

Software Toolchain & Execution

C to Verilog Memory Flow

To execute complex software on our custom RV32I processor, we developed a bare-metal software compilation toolchain. This system translates standard C code into a Verilog memory

initialization file (`.mem`) that can be loaded directly into the FPGA Block RAM. The flow consists of the following components:

- **Memory Mapping (`link.ld`):** A custom linker script maps the processor's memory space, defining a 32KB RAM section originating at address `0x00000000`. It ensures the startup assembly (`.text.init`) is placed at the very beginning of the boot address and defines the top of the RAM (`__stack_top`) to initialize the stack pointer.
- **Startup Assembly (`crt0.S`):** This crucial boot code initializes the stack pointer (`sp`), configures the `mtvec` register to point to our C-based trap handler, enables machine-level interrupts (`mie` and `mstatus`), and ultimately jumps to the C `main()` function.
- **Hardware Abstraction (`main.c`):** We mapped our physical hardware addresses to C pointers using macros (e.g., mapping the UART Base to `0x20000000` and GPIO Base to `0x20001000`). This allowed us to write readable software drivers for UART polling and GPIO mirroring directly in C.
- **Compilation and Conversion (`Makefile`):** We utilized the `riscv64-elf-gcc` cross-compiler to compile the C and Assembly files into an ELF binary, targeting our specific `-march=rv32im` architecture. Finally, we used the `riscv64-elf-objcopy -O verilog` command to extract the compiled machine code into a hexadecimal `firmware.mem` file. This file is read by Vivado's `$readmemh` task to initialize the BRAM before execution.

Software Verification: Fibonacci Execution

A basic Fibonacci program was hardcoded into a `.mem` file and loaded into the processor's instruction memory. During simulation, the processor successfully and accurately computed the Fibonacci sequence.

ASIC Implementation

Design Flow

The physical design flow was executed using the Cadence toolchain.

- **Synthesis (Cadence Genus):** The RTL is synthesized into a gate-level netlist using Cadence Genus. This step maps the Verilog logic to standard cells based on timing and clock constraints defined in the `constraints.sdc` file.
- **Place and Route (Cadence Innovus):** The synthesized netlist is passed to Cadence Innovus for the physical design flow. This step handles floorplanning, power planning, standard cell placement, clock tree synthesis, and routing, ultimately outputting the final layout and Standard Delay Format (SDF) files.

File Structure (VLSI Lab)

The project directory is organized to isolate different stages of the ASIC flow:

- **Verilog/:** Contains the pre-synthesis RTL source files, testbenches, and the `run-iverilog.sh` script for initial simulation.
- **Synthesis/:** Houses the synthesis execution script (`genus_synthesis.sh`), timing constraints (`constraints.sdc`), the Tcl script (`synthesis.tcl`), and the resulting gate-level netlists and reports.

- **physical_design/:** Contains the Innovus execution scripts, the saved layout database (soc_top_asic_done.enc), and the final post-route reports (DRC, Antenna, Connectivity, Area, and Power).
- **Verification/:** Contains the testbenches and the run-incisive.sh script used to simulate the gate-level netlist after synthesis and Place & Route.

Simulation Instructions

To reproduce the ASIC simulations and physical design layout, execute the following steps in the VLSI Lab environment:

- **Step 1 (Pre-Synthesis):** Navigate to the Verilog/ directory and run ./run-iverilog.sh. View the results by opening the generated VCD file using gtkwave waveforms.vcd &.
- **Step 2 (Synthesis):** Navigate to the Synthesis/ directory and execute ./genus_synthesis.sh to generate the gate-level netlist.
- **Step 3 (Place and Route):** Execute the provided Innovus script to complete the physical design flow. To view the final generated layout, launch the Innovus GUI from the terminal and execute: innovus 1> restoreDesign physical_design/soc_top_asic_done.enc.
- **Step 4 (Post-Layout Simulation):** Navigate to the Verification/ directory, verify the target design name inside the shell script, and run ./run-incisive.sh to verify the post-physical design RTL against the original testbench.

Post PnR Results

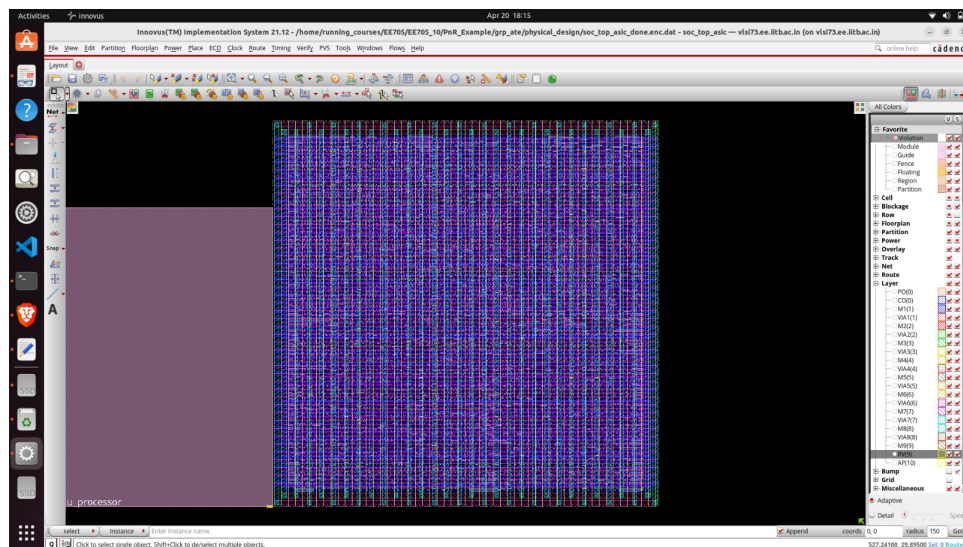


Figure 3: Post PnR Layout

Hinst Name	Module Name	Inst Count	Total Area
soc_top_asic		16352	75509.280
u_processor	processor	16326	75481.200
u_processor/u_control_plane	control_plane	210	1031.400
u_processor/u_control_plane/u_csr_file	csr_file	204	1015.200
u_processor/u_control_plane/u_csr_file/u_csr_mcycle	csr_mcycle	65	409.320
u_processor/u_control_plane/u_csr_file/u_csr_minstret	csr_minstret	98	492.840
u_processor/u_control_plane/u_hazard_unit	hazard_unit	6	16.200
u_processor/u_datapath	datapath	8196	33549.480
u_processor/u_datapath/u_ex_stage	ex_stage	3514	15000.120
u_processor/u_datapath/u_ex_stage/u_alu	alu	811	2586.600
u_processor/u_datapath/u_ex_stage/u_alu_a_mux	ex_alu_a_mux	146	481.680
u_processor/u_datapath/u_ex_stage/u_alu_b_mux	ex_alu_b_mux	158	482.400
u_processor/u_datapath/u_ex_stage/u_branch_comparator	branch_comparator	7	20.160
u_processor/u_datapath/u_ex_stage/u_ex_result_mux	ex_result_mux	107	336.960
u_processor/u_datapath/u_ex_stage/u_id_ex_reg	id_ex_reg	400	1923.480
u_processor/u_datapath/u_ex_stage/u_multiplier	multiplier	1721	8728.560
u_processor/u_datapath/u_ex_stage/u_store_data_mux	store_data_mux	109	337.680
u_processor/u_datapath/u_forwarding_unit	forwarding_unit	168	467.280
u_processor/u_datapath/u_id_stage	id_stage	3836	14961.600
u_processor/u_datapath/u_id_stage/u_control_unit	control_unit	58	126.720
u_processor/u_datapath/u_id_stage/u_id_rs1_fwd_mux	id_rs1_fwd_mux	32	103.680
u_processor/u_datapath/u_id_stage/u_id_rs2_fwd_mux	id_rs2_fwd_mux	32	103.680
u_processor/u_datapath/u_id_stage/u_if_id_reg	if_id_reg	192	915.120
u_processor/u_datapath/u_id_stage/u_imm_gen	imm_gen	65	145.800
u_processor/u_datapath/u_id_stage/u_regfile	regfile	3452	13557.960
u_processor/u_datapath/u_mem_stage	mem_stage	631	2821.320
u_processor/u_datapath/u_mem_stage/u_ex_mem1_reg	ex_mem1_reg	232	1096.200
u_processor/u_datapath/u_mem_stage/u_load_mask_unit	load_mask_unit	57	153.000
u_processor/u_datapath/u_mem_stage/u_mem1_mem2_reg	mem1_mem2_reg	153	729.360
u_processor/u_datapath/u_mem_stage/u_mem2_mem3_reg	mem2_mem3_reg	152	728.280
u_processor/u_datapath/u_mem_stage/u_mem3_wb_mux	mem3_wb_mux	33	105.840
u_processor/u_datapath/u_wb_stage	wb_stage	40	281.880
u_processor/u_datapath/u_wb_stage/u_mem3_wb_reg	mem3_wb_reg	40	281.880
u_processor/u_fetch_unit	fetch_unit	1149	5650.200
u_processor/u_fetch_unit/u_fetch_queue	fetch_queue	963	4294.440
u_processor/u_memory_subsystem	memory_subsystem	6749	35069.400
u_processor/u_memory_subsystem/u_gpio_dummy	gpio_dummy	30	123.120
u_processor/u_memory_subsystem/u_mem_arbiter	mem_arbiter	50	135.720
u_processor/u_memory_subsystem/u_mmio_bridge	mmio_bridge	299	1066.680
u_processor/u_memory_subsystem/u_sha256_accel	sha256_accel	6354	33681.240
u_processor/u_memory_subsystem/u_uart_dummy	uart_dummy	11	33.480

Figure 4: Area Report

DRC & LVS

#Up-Via Summary (total 147229):			
#	single-cut	multi-cut	Total
# M1	70168 (99.9%)	90 (0.1%)	70258
# M2	55396 (100.0%)	0 (0.0%)	55396
# M3	16628 (100.0%)	0 (0.0%)	16628
# M4	4367 (100.0%)	0 (0.0%)	4367
# M5	580 (100.0%)	0 (0.0%)	580
#	147139 (99.9%)	90 (0.1%)	147229
#Total number of DRC violations = 0			
#Total number of process antenna violations = 0			
#Total number of net violated process antenna rule = 0			
#			

Figure 5: DRC and LVS Clean Results

VCD File

Google Drive Link: https://drive.google.com/drive/folders/1X7JXnkF--_1zPS2gTJEgl5uRRXqM_5-I?usp=sharing

FPGA to ASIC Modifications

To port our design from the PYNQ-Z2 FPGA to the Cadence ASIC flow, the core processor logic remained the same, but we had to make a few essential interface changes:

- **ASIC Top Wrapper:** We created a new top-level file (`soc_top_asic.v`) to strip out FPGA-specific board elements and expose standard digital pins for the ASIC layout.
- **Constraints Update:** We swapped out the Xilinx Vivado constraints file (`.xdc`) for standard Synopsys Design Constraints (`.sdc`) to define our clock speeds and pin delays for Cadence Genus and Innovus.

Issues and Bugs

ILA doesn't trigger even though bitstream generation and post-implementation works

Limitations and Incomplete Features

We could not integrate the peripherals and accelerator due to time constraints and independent approach on the design. The design practices were quite different and thus we did not end up working on the same design in detail, and had two different designs (one of which we demo'd)

Individual Contributions

A shared discussion over the design was done, then the verilog implementation was done independently

- Neil: Core RISC-V units: Reg file, Hazard, Control, CSR registers, Pipeline registers, pipeline stages
- Akshat: SHA-256 accelerator, UART, GPIO, MMIO, Multiplier, Secondary core RISC-V unit written in system verilog for cleaner design (we didn't use this for final demo), C to .mem file workflow for running programs

Another more clean implementation

Since the team was working independently we have two versions of nearly complete processors, each of which aren't polished to the brim. The code files for the other one, which also does about the same stuff are included. The other project was written in system verilog and tried to follow as clean code practices as possible. The timing constraints for the project made this design on hold. The files for this are in `proc2` folder, the `README.md` lists all that is needed to run this.

References

- RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture. Version 20260120, 20 Jan. 2026, docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf